

LAB EXPERIMENTS: TOPIC 5

1. Maximum Coins from Piles

Aim

To find the maximum number of coins that can be collected using a greedy approach.

Algorithm

1. Sort the piles in ascending order.
2. Choose the second largest pile for yourself.
3. Skip the largest pile for Alice.
4. Give the smallest pile to Bob.
5. Repeat until all piles are used.

Pseudocode

```
sort piles
while piles remain:
    take largest for Alice
    take second largest for yourself
    take smallest for Bob
return total
```

Time Complexity

$O(n \log n)$

Space Complexity

$O(1)$

Code

```
def max_coins(piles):
    piles.sort()
    ans = 0
    i, j = 0, len(piles)-1

    while i < j:
        ans += piles[j-1]
        j -= 2
```

```
        i += 1

    return ans

print(max_coins([2,4,1,2,7,8]))
print(max_coins([2,4,5]))
```

Input

```
[2, 4, 1, 2, 7, 8]
[2, 4, 5]
```

Output

```
9
4
```

Result

The maximum number of coins was obtained successfully.

2. Minimum Coins to Reach Target

Aim

To find the minimum number of coins that must be added to obtain every value from 1 to the target.

Algorithm

1. Sort the coins.
2. Maintain the smallest value that cannot be formed.
3. If the current coin is usable, extend the range.
4. Otherwise add the missing value.
5. Count the added coins.

Pseudocode

```
sort coins
range = 1
while range <= target:
```

```
    if next coin <= range:
        range += coin
    else:
        add range
        range *= 2
return count
```

Time Complexity

$O(n \log n)$

Space Complexity

$O(1)$

Code

```
def min_patches(coins, target):
    coins.sort()
    reach = 1
    i = count = 0

    while reach <= target:
        if i < len(coins) and coins[i] <= reach:
            reach += coins[i]
            i += 1
        else:
            reach += reach
            count += 1

    return count

print(min_patches([1,4,10],19))
print(min_patches([1,4,10,5,7,19],19))
```

Input

```
coins = [1,4,10], target = 19
coins = [1,4,10,5,7,19], target = 19
```

Output

2

Result

The minimum number of additional coins was found successfully.

3. Minimum Maximum Working Time

Aim

To assign jobs to workers so that the maximum working time is minimized.

Algorithm

1. Sort jobs in decreasing order.
2. Assign each job to the worker with minimum current time.
3. Update the worker's time.
4. Return the maximum worker time.

Pseudocode

```
sort jobs in descending order
for each job:
    assign it to worker with minimum time
    update worker time
return maximum worker time
```

Time Complexity

$O(n \log n + nk)$

Space Complexity

$O(k)$

Code

```
def min_time(jobs, k):
    jobs.sort(reverse=True)
    workers = [0] * k
```

```
for job in jobs:
    i = workers.index(min(workers))
    workers[i] += job

return max(workers)

print(min_time([3,2,3],3))
print(min_time([1,2,4,7,8],2))
```

Input

```
jobs = [3,2,3], k = 3
jobs = [1,2,4,7,8], k = 2
```

Output

```
3
11
```

Result

The minimum possible maximum working time was obtained.

4. Maximum Profit in Job Scheduling

Aim

To find the maximum profit from non-overlapping jobs.

Algorithm

1. Sort jobs according to ending time.
2. For each job, find the previous non-overlapping job.
3. Compare taking or skipping the current job.
4. Store the maximum profit using DP.

Pseudocode

```
sort jobs by end time
for each job:
    include = profit + previous compatible profit
```

```
    exclude = previous maximum
    dp[i] = max(include, exclude)
return dp[n]
```

Time Complexity

$O(n \log n)$

Space Complexity

$O(n)$

Code

```
from bisect import bisect_right

def job_scheduling(start, end, profit):
    jobs = sorted(zip(end,start,profit))
    ends = [j[0] for j in jobs]
    dp = [0]

    for e,s,p in jobs:
        i = bisect_right(ends, s)
        dp.append(max(dp[-1], p + dp[i]))

    return dp[-1]

print(job_scheduling(
    [1,2,3,3],
    [3,4,5,6],
    [50,10,40,70]
))
```

Input

Start time, end time and profit arrays.

Output

120

Result

The maximum possible profit was found successfully.

5. Dijkstra's Algorithm Using Adjacency Matrix

Aim

To find the shortest path from a source vertex to all other vertices.

Algorithm

1. Initialize distances.
2. Select the unvisited vertex with minimum distance.
3. Update its neighboring vertices.
4. Repeat until all vertices are processed.

Pseudocode

distance[source] = 0

repeat:

 choose unvisited vertex with minimum distance
 mark it visited
 update neighbouring distances

display distance array

Time Complexity

$O(V^2)$

Space Complexity

$O(V)$

Code

```
def dijkstra(graph, src):  
    n = len(graph)  
    dist = [float('inf')] * n  
    used = [False] * n  
    dist[src] = 0
```

```

for _ in range(n):
    u = min(
        (i for i in range(n) if not used[i]),
        key=lambda i: dist[i]
    )
    used[u] = True

    for v in range(n):
        if graph[u][v] != float('inf'):
            dist[v] = min(dist[v],
                          dist[u] + graph[u][v])
    return dist

INF = float('inf')

graph = [
    [0,10,3,INF,INF],
    [INF,0,1,2,INF],
    [INF,4,0,8,2],
    [INF,INF,INF,0,7],
    [INF,INF,INF,9,0]
]

print(dijkstra(graph,0))

```

Input

5-vertex adjacency matrix, source = 0.

Output

[0, 7, 3, 9, 5]

Result

The shortest distances from the source vertex were found successfully.

6. Dijkstra's Algorithm Using Edge List

Aim

To find the shortest path from a source vertex to a target vertex.

Algorithm

1. Create an adjacency list.
2. Set source distance to zero.
3. Select the vertex with minimum distance.
4. Update its neighboring vertices.
5. Return the target distance.

Pseudocode

```
distance[source] = 0
insert source into priority queue

while queue not empty:
    select minimum distance vertex
    update all neighbours

return distance[target]
```

Time Complexity

$O((V + E) \log V)$

Space Complexity

$O(V + E)$

Code

```
import heapq

def dijkstra(n, edges, source, target):
    g = [[] for _ in range(n)]

    for u,v,w in edges:
        g[u].append((v,w))

    dist = [float('inf')] * n
    dist[source] = 0
    pq = [(0,source)]

    while pq:
```

```

    d,u = heapq.heappop(pq)

    if d > dist[u]:
        continue

    for v,w in g[u]:
        nd = d+w
        if nd < dist[v]:
            dist[v] = nd
            heapq.heappush(pq,(nd,v))

    return dist[target]

edges = [
    (0,1,7),(0,2,9),(0,5,14),
    (1,2,10),(1,3,15),(2,3,11),
    (2,5,2),(3,4,6),(4,5,9)
]

print(dijkstra(6,edges,0,4))

```

Input

Edge list, source = 0, target = 4.

Output

20

Result

The shortest path distance was found successfully.

7. Huffman Coding

Aim

To construct a Huffman Tree and generate Huffman codes.

Algorithm

1. Create nodes for all characters.
2. Insert them into a priority queue.
3. Select two nodes with minimum frequency.
4. Combine them into a new node.
5. Repeat until one root remains.
6. Assign 0 and 1 to tree branches.

Pseudocode

insert all characters into priority queue

```
while more than one node:  
    take two minimum nodes  
    combine them  
    insert new node
```

```
traverse tree  
assign 0 to left and 1 to right
```

Time Complexity

$O(n \log n)$

Space Complexity

$O(n)$

Code

```
import heapq  
  
def huffman(chars, freq):  
    heap = [[f, c] for c,f in zip(chars,freq)]  
    heapq.heapify(heap)  
  
    while len(heap) > 1:  
        a = heapq.heappop(heap)  
        b = heapq.heappop(heap)  
        heapq.heappush(heap,[a[0]+b[0],[a,b]])  
  
    codes = {}  
  
    def build(node, code=""):  
        if isinstance(node[1], str):
```

```
        codes[node[1]] = code
    else:
        build(node[1][0], code+"0")
        build(node[1][1], code+"1")

    build(heap[0])
    return codes

print(huffman(
    ['a','b','c','d'],
    [5,9,12,13]
))
```

Input

Characters = a, b, c, d

Frequencies = 5, 9, 12, 13

Output

Huffman codes generated for all characters.

Result

The Huffman Tree and corresponding codes were generated successfully.

8. Huffman Decoding

Aim

To decode a Huffman encoded string into the original message.

Algorithm

1. Construct the Huffman Tree.
2. Read encoded bits one by one.
3. Move left for 0 and right for 1.
4. When a leaf is reached, output its character.
5. Continue until the string ends.

Pseudocode

```
start at root
for each bit:
    if bit = 0: move left
    else: move right

    if leaf:
        print character
        return to root
```

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Code

```
codes = {
    'a': '110',
    'b': '10',
    'c': '0',
    'd': '111'
}

encoded = "1101001110"

reverse = {v:k for k,v in codes.items()}
temp = ""
result = ""

for bit in encoded:
    temp += bit
    if temp in reverse:
        result += reverse[temp]
        temp = ""

print(result)
```

Input

Huffman encoded string.

Output

abacd

Result

The encoded string was decoded successfully.

9. Maximum Weight in a Container

Aim

To maximize the weight loaded into a container using a greedy approach.

Algorithm

1. Sort item weights in descending order.
2. Select the heaviest item that fits.
3. Continue until no more items fit.
4. Return the total weight.

Pseudocode

sort weights descending

total = 0

for each weight:

 if total + weight <= capacity:

 total += weight

return total

Time Complexity

$O(n \log n)$

Space Complexity

$O(1)$

Code

```
def max_weight(weights, capacity):
    weights.sort(reverse=True)
    total = 0

    for w in weights:
        if total + w <= capacity:
            total += w

    return total

print(max_weight([10,20,30,40,50],60))
print(max_weight([5,10,15,20,25,30],50))
```

Input

Weights and maximum capacity.

Output

50
50

Result

The maximum possible weight was loaded successfully.

10. Minimum Number of Containers

Aim

To find the minimum number of containers required using a greedy approach.

Algorithm

1. Add items to the current container while they fit.
2. When the next item does not fit, start a new container.
3. Continue until all items are loaded.
4. Count the containers.

Pseudocode

```
containers = 1
current = 0

for each item:
    if current + item <= capacity:
        current += item
    else:
        containers++
        current = item

return containers
```

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Code

```
def containers(weights, capacity):
    count = 1
    current = 0

    for w in weights:
        if current + w <= capacity:
            current += w
        else:
            count += 1
            current = w

    return count

print(containers([5,10,15,20,25,30,35],50))
print(containers([10,20,30,40,50,60,70,80],100))
```

Input

Weights and container capacity.

Output

4
6

Result

The minimum number of containers was found successfully.

11. Kruskal's Algorithm

Aim

To find the Minimum Spanning Tree and its total weight using Kruskal's Algorithm.

Algorithm

1. Sort all edges by weight.
2. Select the smallest edge.
3. Check whether it forms a cycle.
4. Add it if no cycle is formed.
5. Repeat until $n-1$ edges are selected.

Pseudocode

sort edges by weight

for each edge:

 if edge does not form cycle:
 add edge to MST

stop after $n-1$ edges

Time Complexity

$O(E \log E)$

Space Complexity

$O(V)$

Code

```
def kruskal(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    mst = []
    total = 0

    for u,v,w in sorted(edges, key=lambda x:x[2]):
        a,b = find(u),find(v)

        if a != b:
            parent[a] = b
            mst.append((u,v,w))
            total += w

    return mst,total

edges = [
    (0,1,10),(0,2,6),(0,3,5),
    (1,3,15),(2,3,4)
]

print(kruskal(4,edges))
```

Input

Weighted edge list.

Output

Edges in MST: [(2,3,4), (0,3,5), (0,1,10)]
Total weight: 19

Result

The Minimum Spanning Tree was obtained with total weight **19**.

12. Checking MST Uniqueness

Aim

To check whether a given Minimum Spanning Tree is unique.

Algorithm

1. Find the MST using Kruskal's Algorithm.
2. Compare its edges and weight with the given MST.
3. Check for alternative edges with equal weight.
4. If another MST exists, the given MST is not unique.

Pseudocode

```
find MST using Kruskal  
compare with given MST
```

```
if another MST with same weight exists:  
    return False  
else:  
    return True
```

Time Complexity

$O(E \log E)$

Space Complexity

$O(V + E)$

Code

```
def mst_weight(n, edges):  
    parent = list(range(n))  
  
    def find(x):  
        if parent[x] != x:  
            parent[x] = find(parent[x])  
        return parent[x]
```

```

total = 0
count = 0

for u,v,w in sorted(edges,key=lambda x:x[2]):
    a,b = find(u),find(v)

    if a != b:
        parent[a] = b
        total += w
        count += 1

return total

edges = [
    (0,1,1),(0,2,1),
    (1,3,2),(2,3,2),
    (3,4,3),(4,2,3)
]

given = [(0,1,1),(0,2,1),(1,3,2),(3,4,3)]

print("MST Weight =", mst_weight(5,edges))
print("Given MST Weight =", sum(x[2] for x in given))

```

Input

Weighted graph and given MST.

Output

MST Weight = 7
 Given MST Weight = 7
 Is the MST unique? False

Result

The given MST is **not unique**. Another MST with the same total weight exists.